

计算机中的常见码制及转换部分复习题

一、数制的基本概念

1. **定义**：用一组固定的数字和一套统一的规则来表示数的方法。
2. **基数**：数制中所使用数字的个数。例如十进制基数为10，二进制基数为2。
3. **位权**：数制中某一位上的“1”所表示的数值大小，是基数的幂次。例如十进制数 555，第一个5的位权是 $10^2=100$ 。
4. **常用数制**：
 - **十进制**：基数为10，逢十进一。人类常用。
 - **二进制**：基数为2，逢二进一。计算机内部使用。
 - **八进制**：基数为8，逢八进一。作为二进制的缩写。
 - **十六进制**：基数为16，逢十六进一。作为二进制的缩写。

二、为什么计算机使用二进制？

1. **技术实现简单**：二进制只有0和1两个状态，容易用物理量表示（如电压高低、晶体管通断），稳定可靠。
2. **运算规则简单**：加法和乘法规则各只有4条，简化了计算机运算器的设计。
3. **便于逻辑运算**：二进制的0和1可对应逻辑代数中的“假”和“真”，便于进行逻辑判断。
4. **抗干扰能力强**：识别两个稳定状态比识别多个状态更可靠。

三、数制间的转换（核心重点）

1. 非十进制数 → 十进制数

- **方法**：按权展开相加。
- **公式**： $(A_n...A_1A_0.A_{-1}...A_{-m})_N = \sum_{i=n}^{-m} A_i \times N^i$
- **例题1**：将二进制数 $(1011.01)_2$ 转换为十进制。
 - **解**：
$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 8 + 0 + 2 + 1 + 0 + 0.25 = 11.25$$
- **习题变形**：将八进制数 $(143.64)_8$ 转换为十进制。

◦ 解：

$$1 \times 8^2 + 4 \times 8^1 + 3 \times 8^0 + 6 \times 8^{-1} + 4 \times 8^{-2} = 64 + 32 + 3 + 0.75 + 0.0625 = 99.8125$$

2. 十进制数 $\rightarrow$ 非十进制数

• 方法：整数部分和小数部分分别转换。

◦ 整数部分：除基取余，逆序排列。直到商为0为止。

◦ 小数部分：乘基取整，顺序排列。直到小数部分为0或达到所需精度。

• 例题2：将十进制数 $(125.6875)_{10}$ 转换为二进制。

◦ 整数部分(125)转换：

Plain Text

```
1 125 ÷ 2 = 62 ... 余 1 (最低位)
2 62 ÷ 2 = 31 ... 余 0
3 31 ÷ 2 = 15 ... 余 1
4 15 ÷ 2 = 7 ... 余 1
5 7 ÷ 2 = 3 ... 余 1
6 3 ÷ 2 = 1 ... 余 1
7 1 ÷ 2 = 0 ... 余 1 (最高位)
```

$$\therefore (125)_{10} = (1111101)_2$$

◦ 小数部分(0.6875)转换：

Plain Text

```
1 0.6875 × 2 = 1.375 ... 取整 1 (最高位)
2 0.375 × 2 = 0.75 ... 取整 0
3 0.75 × 2 = 1.5 ... 取整 1
4 0.5 × 2 = 1.0 ... 取整 1 (最低位)
```

$$\therefore (0.6875)_{10} = (0.1011)_2$$

◦ 合并： $(125.6875)_{10} = (1111101.1011)_2$

• 习题变形：将十进制数 $(25.25)_{10}$ 转换为二进制。

◦ 整数部分(25)： $25 \div 2 = 12 \dots 1$, $12 \div 2 = 6 \dots 0$, $6 \div 2 = 3 \dots 0$, $3 \div 2 = 1 \dots 1$, $1 \div 2 = 0 \dots 1 \rightarrow (11001)_2$

- **小数部分(0.25):** $0.25 \times 2 = 0.5 \dots 0$, $0.5 \times 2 = 1.0 \dots 1 \rightarrow (0.01)_2$
- **合并:** $(25.25)_{10} = (11001.01)_2$

3. 二进制 $\leftrightarrow$ 八进制/十六进制

- **方法:** 以小数点为界, 向左向右分组。
 - **二进制 $\rightarrow$ 八进制:** 3位一组, 不足补零。因为 $2^3 = 8$ 。
 - **二进制 $\rightarrow$ 十六进制:** 4位一组, 不足补零。因为 $2^4 = 16$ 。
 - **八进制/十六进制 $\rightarrow$ 二进制:** 将每一位数展开成3位或4位二进制数。
- **例题3:** 将二进制数 $(1100101.0101)_2$ 转换为八进制。
 - **解:** 分组: $001\ 100\ 101\ .\ 010\ 100$ (不足位补零) $\rightarrow$ 对应八进制数为 $1\ 4\ 5\ .\ 2\ 4 \rightarrow (145.24)_8$
- **例题4:** 将二进制数 $(101101110.1001001)_2$ 转换为十六进制。
 - **解:** 分组: $0001\ 0110\ 1110\ .\ 1001\ 0010 \rightarrow$ 对应十六进制数为 $1\ 6\ E\ .\ 9\ 2 \rightarrow (16E.92)_{16}$
- **习题变形:** 将十六进制数 $(5A0B.0C)_{16}$ 转换为二进制。
 - **解:** $5 \rightarrow 0101$, $A \rightarrow 1010$, $0 \rightarrow 0000$, $B \rightarrow 1011$, $0 \rightarrow 0000$, $C \rightarrow 1100 \rightarrow (101101000001011.00001100)_2$

四、带符号数的机器表示 (原码、反码、补码)

为解决负数在计算机中的表示和运算问题, 引入了机器数。

1. 原码

- **定义:** 最高位为符号位 (0正1负), 其余位表示数值的绝对值。
- **优点:** 简单直观。
- **缺点:**
 - **存在“+0”和“-0”:** $[+0]_{\text{原}} = 00000000$, $[-0]_{\text{原}} = 10000000$ 。
 - **加减运算复杂:** 不能直接带符号位运算。
- **例题5:** 用8位二进制表示+73和-73的原码。
 - $+73 = (1001001)_2 \rightarrow [+73]_{\text{原}} = 01001001$
 - $[-73]_{\text{原}} = 11001001$

2. 反码

- 定义：
 - 正数：反码 = 原码。
 - 负数：符号位不变，其余位按位取反。
- 作用：作为求补码的中间码。解决了原码的部分运算问题，但“0”的表示仍不唯一。

3. 补码（重点中的重点）

- 定义：
 - 正数：补码 = 原码。
 - 负数：补码 = 反码 + 1。
- 优点：
 - 解决了“0”的唯一表示： $[0]_{\text{补}} = 00000000$ 。
 - 可将减法运算转换为加法运算： $[X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$ 。
- 表示范围：对于n位二进制，补码表示范围为 -2^{n-1} 到 $2^{n-1} - 1$ 。
- 例题6：求-67的8位补码。
 - $[+67]_{\text{原}} = 01000011$
 - $[-67]_{\text{原}} = 11000011$
 - $[-67]_{\text{反}} = 10111100$ （符号位不变，数值位取反）
 - $[-67]_{\text{补}} = 10111101$ （反码+1）
- 习题变形：已知 $[X]_{\text{补}} = 10101100$ ，求X的真值。
 - 解：最高位为1，是负数。
 - 补码求原码：10101100 (补) → 11010011 (反) → 11010100 (原) （方法：补码取反加1或减1取反）
 - $\therefore \text{真值} = -(1 \times 2^6 + 1 \times 2^4 + 1 \times 2^2) = -(64 + 16 + 4) = -84$ 。

五、信息的编码

1. BCD码

- 定义：用4位二进制数表示1位十进制数。
- 8421码：最常用，4位二进制数的位权分别是8,4,2,1。
- 例题7：1995的8421码是 0001 1001 1001 0101。

2. ASCII码

- 定义：美国标准信息交换码，用7位二进制表示一个字符。
- 重要字符范围：
 - 数字 ‘0’ - ‘9’：00110000 (30H) 到 00111001 (39H)
 - 大写字母 ‘A’ - ‘Z’：01000001 (41H) 到 01011010 (5AH)
 - 小写字母 ‘a’ - ‘z’：01100001 (61H) 到 01111010 (7AH)

3. 汉字编码

- 输入码：用于输入汉字（如拼音、五笔）。
- 国标码：用于信息交换（如GB2312）。
- 机内码：计算机内部存储和处理汉字的编码。机内码 = 国标码 + 8080H（即将两个字节的最高位都置1，以区分于ASCII码）。
- 字型码：用于汉字输出（点阵码）。

重要缩略语列表

缩略语	英文全称	中文全称
BCD	Binary-Coded Decimal	二进制编码的十进制
ASCII	American Standard Code for Information Interchange	美国标准信息交换码
GB2312	-	汉字编码字符集·基本集
MSB	Most Significant Bit	最高有效位
LSB	Least Significant Bit	最低有效位

本章复习题（附答案详解）

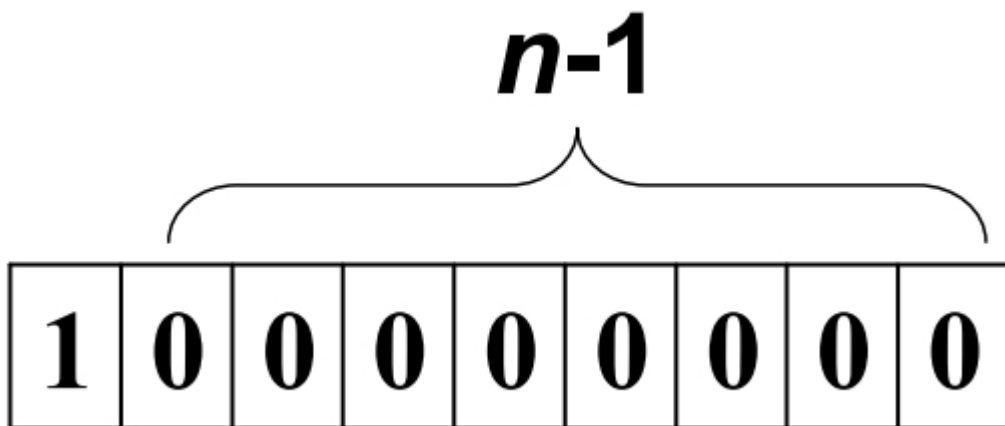
一、选择题

1. 下列数中，最小的数是（ ）。 A. $(101001)_2$ B. $(52)_8$ C. $(2B)_{16}$ D. $(44)_{10}$

详解：将所有数转换为十进制比较。A: 41, B: $5 \times 8 + 2 = 42$, C: $2 \times 16 + 11 = 43$, D: 44。最小的是A: 41。

2. 一个8位二进制整数采用补码表示，其表示的范围是（）。A. -128 到 +127 B. -127 到 +127 C. -128 到 +128 D. -127 到 +128

详解： n 位补码的表示范围是 -2^{n-1} 到 $2^{n-1} - 1$ 。8位时，范围为 -128 到 +127。故选A。



3. 已知 $[X]_{\text{补}} = 10011011$ ，则 $[X]_{\text{原}}$ 为（）。A. 11100101 B. 10011011 C. 00011011 D. 11100100

详解：最高位为1，是负数。由补码求原码：补码 10011011 减1得 10011010，再按位取反（符号位不变）得 11100101。故选A。

二、计算与转换题

1. 将十进制数 $(87.625)_{10}$ 转换为二进制和十六进制。

详解：

整数部分(87)转二进制： $87 \div 2 = 43 \dots 1$, $43 \div 2 = 21 \dots 1$, $21 \div 2 = 10 \dots 1$, $10 \div 2 = 5 \dots 0$, $5 \div 2 = 2 \dots 1$, $2 \div 2 = 1 \dots 0$, $1 \div 2 = 0 \dots 1$ 。逆序排列得 1010111。

小数部分(0.625)转二进制： $0.625 \times 2 = 1.25 \dots 1$, $0.25 \times 2 = 0.5 \dots 0$, $0.5 \times 2 = 1.0 \dots 1$ 。顺序排列得 0.101。

合并： $(87.625)_{10} = (1010111.101)_2$ 。

转十六进制：二进制分组 0101 0111 . 1010 $\rightarrow$ 5 7 . A。 $\therefore (87.625)_{10} = (57.A)_{16}$ 。

2. 已知 $[X]_{\text{补}} = 01011101$ ， $[Y]_{\text{补}} = 10100101$ ，求 $[X + Y]_{\text{补}}$ 并判断是否溢出。

详解：

计算： $01011101 + 10100101 = 100000010$ 。由于是8位补码，最高位（第9位）进位丢弃。 $\therefore [X + Y]_{\text{补}} = 00000010$ 。

判断溢出：采用双高位判别法。 C_s （符号位进位）为0， C_p （最高数值位进位）为1。**溢出** = $C_s \oplus C_p = 0 \oplus 1 = 1$ ，表示有溢出。

结论：结果溢出（因为正数 + 负数 = 很小的正数，结果不合理）。

三、简答题

1. 简述补码的设计目的和优点。

答案要点：

目的：解决原码和反码在表示“0”和进行加减运算时存在的缺陷。

优点：

0的表示唯一： $[+0]_{\text{补}} = [-0]_{\text{补}}$ 。

简化运算：可以将减法运算统一为加法运算，简化了CPU中运算器的设计。符号位可以和数值位一起参与运算。

六、数的定点表示与浮点表示（补充详解）

计算机在处理实数时，主要有两种方法：定点表示法和浮点表示法。

1. 定点表示法

小数点位置固定不变。主要用于表示整数或纯小数。

- **定点纯小数：**小数点固定在数值部分最高位之前。格式为 符号位 . 数值部分。表示范围： $|X| < 1$ 。
- **定点纯整数：**小数点固定在数值部分最低位之后。格式为 符号位 数值部分。表示范围取决于数值部分位数。
- **缺点：**表示范围小，缺乏灵活性。表示一个大数或非常小的数时，需要选择“比例因子”，使用不便。

2. 浮点表示法（重点）

类似于科学计数法，通过改变指数部分使小数点位置浮动，从而可以表示更大范围的数。

- **表示形式：** $N = \pm D \times 2^{\pm E}$

- D : **尾数**，是一个纯小数，决定数的精度。通常进行**规格化**，即尾数的绝对值最高位必须为1（即 $\frac{1}{2} \leq [1](@context - ref?id = 7)|D| < 1$ ）。
- E : **阶码**，是一个整数，决定数的表示范围。
- **机器中的存储格式**：一个浮点数由**阶码部分**和**尾数部分**组成。

阶符	阶码 E	数符	尾数 D
1位	n位	1位	m位

- **阶码部分**：常用**补码**或**移码**表示，看作一个**定点整数**。
- **尾数部分**：常用**补码**或**原码**表示，看作一个**定点小数**。

3. 浮点数表示示例与计算

例题8：一个32位浮点数，阶码8位（含1位阶符），尾数24位（含1位数符），均用补码表示，基数为2。写出存放浮点数105.5的格式。

详解：

- 将十进制数转换为二进制： $105.5 = (1101001.1)_2$
- 规格化：将二进制数表示为 $\pm D \times 2^{\pm E}$ 的形式。 $1101001.1 = 0.11010011 \times 2^7$ （小数点左移7位） $\therefore$ 尾数 $D = +0.11010011$ ，阶码 $E = +7$ 。
- 将阶码和尾数转换为补码：
 - **阶码E(+7)**：原码为 00000111，补码为 00000111。
 - **尾数D(+0.11010011)**：正数的补码等于原码。注意尾数部分占24位，数符为0，所以需要在尾部补0以满足位数： 0.11010011 0000000000000000（共24位，小数点不存储，是隐含的）。
- 组合成浮点数**：阶符(0) 阶码(0000111) | 数符(0) 尾数(110100110000000000000000) 为了方便阅读，通常写成十六进制。将上述二进制串按4位一组转换： 0000 0111 0110 1001 1000 0000 0000 0000 -> 07698000H。 $\therefore$ 105.5在给定格式下的浮点数表示为 07698000H。

例题9（文档变形题）：同上格式，写出存放浮点数-256.5的格式。

详解：

- 转换为二进制： $-256.5 = (-100000000.1)_2$
- 规格化： -0.1000000001×2^9 $\therefore$ 尾数 $D = -0.1000000001$ ，阶码 $E = +9$ 。

3. 转换为补码:

- **阶码E(+9):** 原码 00001001, 补码 00001001。
- **尾数D(-0.1000000001):**
 - 原码: 1.100000000100000000000000 (数符1, 数值位取绝对值)
 - 反码: 1.011111111011111111111111
 - 补码: 1.011111111100000000000000 (反码+1)

4. **组合:** 阶码部分(0 0001001) + 尾数部分(1 011111111100000000000000) 二进制串: 00001001 10111111 11000000 00000000 -> 09BFC000H。 ∴ -256.5的浮点数表示为 09BFC000H。

4. 浮点数的溢出

- **上溢:** 一个数的阶码 > 计算机所能表示的最大阶码。系统会将其视为 $+\infty$ 或 $-\infty$, 通常会导致溢出中断。
- **下溢:** 一个数的阶码 < 计算机所能表示的最小阶码。系统会将其作为**机器零**处理 (即所有位为0), 程序仍可继续运行。

七、汉字编码体系详解 (GB2312-80)

汉字编码解决了汉字在计算机中的输入、存储、处理和输出问题。

1. GB2312-80 国标码 (信息交换用汉字编码字符集·基本集)

- **全称:** 《信息交换用汉字编码字符集·基本集》
- **收录内容:**
 - **一级汉字:** 3755个, 按汉语拼音字母顺序排列。
 - **二级汉字:** 3008个, 按偏旁部首顺序排列。
 - **非汉字字符:** 682个 (包括拉丁字母、希腊字母、日文假名、俄文字母、序号、数字、制表符等)。
 - **总计:** 7445个字符。
- **编码结构:** 采用94×94的矩阵 (区位图)。纵向为“区”, 横向为“位”。
 - **01-09区:** 非汉字字符 (符号、数字、外文等)。
 - **10-15区:** 空白区, 用于自定义符号。

- **16-55区**：一级汉字（最常用的汉字）。
- **56-87区**：二级汉字。
- **88-94区**：空白区。

2. 区位码

- **定义**：用两位十进制数直接表示汉字在GB2312字符集中的位置。第一位是**区号**（01-94），第二位是**位号**（01-94）。
- **示例**：“啊”字位于第16区第01位，其**区位码**就是 1601。

3. 国标码

- **定义**：将区位码的区号和位号分别转换为十六进制后，再各自加上 20H（32D）得到的编码。
- **目的**：避开ASCII码中的前32个控制字符和空格符（20H），保证每个字节的值都大于32（即 21H ~ 7EH），与可打印的ASCII码兼容。
- **转换公式**：国标码 = 区位码（十六进制） + 2020H

4. 机内码

- **定义**：汉字在计算机内部进行存储、处理和传输时使用的编码。
- **产生原因**：国标码的每个字节范围（21H-7EH）与可打印ASCII码重叠。如果直接使用，计算机无法区分一个字节 41H 到底是字母 ‘A’ 还是某个汉字的一部分。因此，将国标码的每个字节的**最高位由0变为1**，这就是机内码。
- **转换公式**：机内码 = 国标码 + 8080H 或 机内码 = 区位码（十六进制） + A0A0H

5. 转换关系总结与例题

转换链条：区位码 --(+2020H)--> 国标码 --(+8080H)--> 机内码

例题10（文档原题）：一个汉字的国标码是 4E52H，则其机内码为？

详解：

- **方法**：机内码 = 国标码 + 8080H
- **计算**：4E52H + 8080H = CED2H
- **答**：其机内码为 CED2H。

例题11（综合题）：汉字“保”的国标码为 3123H，求它的区位码和机内码。

详解：

1. 求区位码：

- 区位码 = 国标码 - 2020H
- $3123H - 2020H = 1103H$
- 将 1103H 转换为十进制：区号 11H = 17D，位号 03H = 03D。
- $\therefore$ 区位码为 1703。

2. 求机内码：

- 机内码 = 国标码 + 8080H
- $3123H + 8080H = B1A3H$
- (或使用区位码计算：机内码 = 区位码(十六进制) + A0A0H = 1103H + A0A0H = B1A3H)
- $\therefore$ 机内码为 B1A3H。

本章补充复习题（附答案详解）

1. 浮点数计算题

某浮点数格式如下：阶码6位（含1位阶符，补码表示），尾数10位（含1位数符，补码表示），基数为2。请将十进制数 -12.75 表示为该格式的浮点数。

详解：

1. 转换为二进制： $-12.75 = (-1100.11)_2$

2. 规格化： -0.110011×2^4 （小数点左移4位）

3. 求补码：

- 阶码E(+4)：6位补码为 000100。
- 尾数D(-0.110011)：
 - 原码：1.110011000 （尾数共10位，数符1，数值位补0）
 - 反码：1.001100111
 - 补码：1.001101000 （反码末位+1）

4. 组合：阶码(000100) + 尾数(1001101000) -> 二进制串 0001001001101000。 $\therefore$ 该浮点数表示为 0001001001101000B。

2. 汉字编码转换题

已知汉字“啊”的区位码是 1601，请计算它的国标码和机内码（用十六进制表示）。

详解：

1. 将区位码转换为十六进制：区号16D = 10H，位号01D = 01H。∴ 区位码(十六进制) = 1001H。
2. 求国标码：国标码 = 区位码(十六进制) + 2020H = 1001H + 2020H = 3021H。
3. 求机内码：机内码 = 国标码 + 8080H = 3021H + 8080H = B0A1H。（或：机内码 = 区位码(十六进制) + A0A0H = 1001H + A0A0H = B0A1H）答：国标码为 3021H，机内码为 B0A1H。

八、数据校验码：奇偶校验码（补充详解）

1. 基本概念

- **目的**：检测数据在传输或存储过程中是否发生了**单个比特的错误**（1位错）。
- **核心思想**：通过增加一个额外的**校验位**，使整个码字（数据位 + 校验位）中“1”的个数满足特定的奇偶性。
- **分类**：
 - **奇校验**：添加校验位后，整个码字中“1”的个数为**奇数**。
 - **偶校验**：添加校验位后，整个码字中“1”的个数为**偶数**。

2. 工作流程与例题

工作流程：

1. **发送方**：根据要发送的原始数据位中“1”的个数，计算并添加一个校验位，形成传输码字。
2. **传输**：发送码字。
3. **接收方**：接收码字，并计算其中“1”的个数。
4. **判断**：
 - 如果“1”的个数与约定的奇偶性一致，则认为传输正确。
 - 如果不一致，则断定传输过程中发生了**奇数个比特错误**（最可能的是1位错）。

例题12（垂直/纵向奇偶校验）：假设采用**偶校验**，为7位数据 1011001 添加奇偶校验位。

详解：

1. **计算数据位中“1”的个数**：数据 1011001 中有4个“1”（偶数个）。
2. **根据偶校验规则添加校验位**：偶校验要求整个码字中“1”的个数为偶数。现有数据位已有偶数个“1”，因此校验位应设置为 0，使总数保持偶数。

3. 形成传输码字：数据位(7位) + 校验位(1位) = 1011001 + 0 = 10110010。

- 此时码字 10110010 中“1”的个数为4（偶数），符合偶校验规则。

例题13（文档原题变形）：下表为7位数据，采用**奇校验**。请填写校验位 b_8 。

数据位 (b7b6b5b4b3b2b1)	校验位 (b8)
0 1 1 0 0 1 0	?
1 0 1 1 0 0 1	?

详解：

- 第一行数据 0110010：
 - “1”的个数为3（奇数）。
 - 奇校验要求总数为奇数。数据位已有奇数个“1”，所以校验位应为0，使总数保持奇数。
 - $\therefore b_8 = 0$ 。
- 第二行数据 1011001：
 - “1”的个数为4（偶数）。
 - 奇校验要求总数为奇数。数据位是偶数个“1”，所以校验位应为1，使总数变为奇数。
 - $\therefore b_8 = 1$ 。

3. 奇偶校验的三种实现方式

1. 水平奇偶校验码

- 方法：**为**每一个**传输的数据单元（如一个字节）单独添加一个校验位。
- 特点：**实现简单，但对突发错误（连续多位出错）检测能力弱。

2. 垂直奇偶校验码

- 方法：**将一批数据（如多个字节）的**相同位**（所有字节的第0位、第1位...）进行奇偶校验，产生一个校验字节。
- 特点：**能检测出每个垂直列上的奇数个错误。

3. 水平垂直奇偶校验码（方阵校验）

- 方法：**先进行**垂直校验**（为每一列产生一个校验位），然后再对包括垂直校验位在内的所有行进行**水平校验**（为每一行产生一个校验位）。

- **特点：**功能强大。不仅能发现一位错或奇数位错，还能根据行和列校验结果的变化，定位并纠正一位错。例如，如果第*i*行的水平校验出错，同时第*j*列的垂直校验也出错，则可以定位是第*i*行第*j*列的那个比特出错了，并将其取反纠正。

例题14（水平垂直奇偶校验）：假设要传输4个7位数据：1101001, 1011010, 0111100, 1110001。采用偶校验的方阵校验。

详解：

1.

组织数据：将4个7位数据排成4行。

Plain Text

```
1 行1: 1 1 0 1 0 0 1
2 行2: 1 0 1 1 0 1 0
3 行3: 0 1 1 1 1 0 0
4 行4: 1 1 1 0 0 0 1
```

2. **进行垂直校验（计算列校验位）：**对每一列（垂直方向）的4个比特进行偶校验，产生一个校验位，形成第5行（垂直校验行）。

- 第1列：1, 1, 0, 1 -> “1”的个数=3（奇）-> 校验位=1（使总数为偶）
- 第2列：1, 0, 1, 1 -> “1”的个数=3（奇）-> 校验位=1
- 第3列：0, 1, 1, 1 -> “1”的个数=3（奇）-> 校验位=1
- 第4列：1, 1, 1, 0 -> “1”的个数=3（奇）-> 校验位=1
- 第5列：0, 0, 1, 0 -> “1”的个数=1（奇）-> 校验位=1
- 第6列：0, 1, 0, 0 -> “1”的个数=1（奇）-> 校验位=1
- 第7列：1, 0, 0, 1 -> “1”的个数=2（偶）-> 校验位=0
- **垂直校验行：**1 1 1 1 1 1 0

3. **进行水平校验（计算行校验位）：**对每一行（包括垂直校验行）的7个数据位进行偶校验，产生一个校验位，放在第8列。

- 行1 1101001： “1”的个数=4（偶）-> 校验位=0
- 行2 1011010： “1”的个数=4（偶）-> 校验位=0
- 行3 0111100： “1”的个数=4（偶）-> 校验位=0
- 行4 1110001： “1”的个数=4（偶）-> 校验位=0

- 垂直校验行 1111110：“1”的个数=6（偶）-> 校验位=0

4.

最终传输的方阵（5行8列）：

Plain Text

1	1	1	0	1	0	0	1		0
2	1	0	1	1	0	1	0		0
3	0	1	1	1	1	0	0		0
4	1	1	1	0	0	0	1		0
5	1	1	1	1	1	1	0		0

接收方按同样的规则校验行和列。如果只有某一行和某一列的校验出错，即可定位错误位。

4. 奇偶校验码的优缺点

- 优点：

- 实现简单，成本低。
- 能够检测出所有**奇数个比特**的错误。

- 缺点：

- **不能检测偶数个比特的错误**（因为偶数个错误不改变“1”的奇偶总数）。
- **没有纠错能力**（除了水平垂直奇偶校验可以纠正1位错）。它只能告诉你有错误发生，但不知道是哪里错了。

奇偶校验码补充复习题（附答案详解）

1. 基础题

已知采用奇校验。接收方收到的码字为 10110101（最高位为校验位）。请判断传输是否出错。

详解：

1. 计算码字中“1”的个数：10110101 中有5个“1”。
2. 与奇校验规则对比：奇校验要求“1”的个数为奇数。5是奇数，符合规则。
3. 判断：传输**正确**，未检测到错误。

2. 综合题

某通信系统采用**水平垂直偶校验**。发送方发出的部分数据方阵如下（第5行是垂直校验行，第6列是水平校验列）：

Plain Text

1	数据位						水平校验位
2	1	0	1	1	0	1	?
3	0	1	0	1	1	0	?
4	1	1	0	0	1	1	?
5	0	0	1	1	0	0	?
6	?	?	?	?	?	?	? (垂直校验行)

- (1) 请补全表中所有的？（即水平校验位和垂直校验行）。
- (2) 若接收方收到的第2行第3列数据由 0 变成了 1，请描述接收方的检错和纠错过程。

详解：

(1) 补全校验位

- **计算水平校验位（行校验）：**对每一行进行偶校验。
 - 行1 101101：“1”的个数=4（偶）-> 校验位=0
 - 行2 010110：“1”的个数=3（奇）-> 校验位=1（使总数为偶）
 - 行3 110011：“1”的个数=4（偶）-> 校验位=0
 - 行4 001100：“1”的个数=2（偶）-> 校验位=0
- **计算垂直校验行（列校验）：**对每一列（包括刚算出的水平校验列）进行偶校验。
 - 第1列：1,0,1,0 -> “1”的个数=2（偶）-> 校验位=0
 - 第2列：0,1,1,0 -> “1”的个数=2（偶）-> 校验位=0
 - 第3列：1,0,0,1 -> “1”的个数=2（偶）-> 校验位=0
 - 第4列：1,1,0,1 -> “1”的个数=3（奇）-> 校验位=1
 - 第5列：0,1,1,0 -> “1”的个数=2（偶）-> 校验位=0
 - 第6列（水平校验列）：0,1,0,0 -> “1”的个数=1（奇）-> 校验位=1
- **最后计算垂直校验行的水平校验位：**垂直校验行 000101：“1”的个数=2（偶）-> 校验位=0。
-

补全后的方阵：

Plain Text

1	1	0	1	1	0	1		0
2	0	1	0	1	1	0		1
3	1	1	0	0	1	1		0
4	0	0	1	1	0	0		0
5	0	0	0	1	0	1		0

(2) 检错与纠错过程

1. 接收方校验：

- **水平校验：**发现**第2行**的水平校验出错（接收到的行是 011110，“1”的个数=4，是偶数，但校验位是1，预期应为0，不一致）。
- **垂直校验：**发现**第3列**的垂直校验出错（接收到的列是 1, 1, 0, 1，“1”的个数=3，是奇数，但校验位是0，预期应为1，不一致）。

2. 定位错误：水平校验指出第2行错，垂直校验指出第3列错。交点即为错误位：**第2行第3列**。

3. 纠错：将该位数据取反，将接收到的 1 纠正为 0。